

Shootings (shooting)



Siz kompyuteringizda Red Dead Redemption VI o'yinini o'ynamoqdasiz.

Bankda otishma bo'lishiga oz qoldi. n ta odam to'pponcha bilan turibdi, i -odam to'pponchasi $p[i]$ odamga qaratilgan, bunda $p[i] \neq i$. E'tibor bering, barcha $p[i]$ **qiymatlari har xil**.

i -odam to'pponchasidan $t[i]$ vaqtda o'q uzmoqchi. Otilgan odam shu zahoti o'ladi va albatta o'zi otmoqchi bo'lgan vaqtda o'q uza olmaydi.

Undan tashqari, sizga q ta so'rov keladi. Har bir so'rovda qaysidir x -odamning o'q uzish vaqti y ga yangilanishini bildiradi. So'rovlardan oldin va barcha so'rovlar oralig'ida **hech qaysi ikkita odamning o'q uzish vaqtlari bir xil bo'lmaydi**.

Har bir so'rovdan so'ng, agar o'sha holatda otishma boshlanganida necha kishi tirik qolishini toping. Siz kompyuter o'yinini o'ynayapsiz, shuning uchun har bir so'rov tugaganidan so'ng odamlar "qayta tiriladi" deb hisoblashingiz mumkin.

Implementation details

Vazifangiz quyidagi protseduralarni dasturlash:

```
void init(int n, int[] p, int[] t)
```

- n : jami odamlar soni.
- p va t : uzunliklari n ga teng bo'lgan massivlar – har bir odamning kimni otmoqchi ekanligi va bu ishni qachon qilmoqchi ekanligi.
- Bu protsedura aynan bir marta chaqiriladi.

```
int change_time(int x, int y)
```

- x : odamning raqami.
- y : yangilangan vaqt.
- Bu protsedura yangilangan qiymatlar bilan otishma boshlanganida nechta odam tirik qolishini aytishi kerak.

- Bu protsedura aynan q marta chaqiriladi.

Example

Ushbu chaqiruvni ko'raylik:

```
init(3, [1, 2, 3], [0, 1, 2])
```

So'ng:

```
change_time(0, 3)
```

Bunda 0-odamning otish vaqti 3ga o'zgartirildi. Ya'ni hozirda $t = [3, 1, 2]$.

Avval 1-odam o'q uzadi va 2-odam o'ladi. Shunda 2-odam vaqti kelganida o'q uzmaydi va 0-odam tirik qoladi. 0-odam vaqti kelganida 1-odamni o'ldiradi.

Natijada faqat 0-odam tirik qoladi. Shuning uchun bu protsedura 1 qaytarishi kerak.

Keyingi chaqiruvda:

```
change_time(2, 5)
```

Bunda 2-odamning otish vaqti 5ga o'zgartirildi. Ya'ni hozirda $t = [3, 1, 5]$.

Avval 1-odam o'q uzadi va 2-odam o'ladi. So'ng 0-odam o'q uzadi va 1-odam o'ladi. 2-odam vaqti kelganida o'q uzmaydi va 0-odam tirik qoladi.

Natijada yana faqatgina 0-odam tirik qoladi. Shuning uchun bu protsedura ham 1 qaytarishi kerak.

Constraints

- $1 \leq n, q \leq 100\,000$
- $0 \leq p[i] \leq n - 1$ (barcha $0 \leq i \leq n - 1$ uchun)
- $p[i] \neq i$ (barcha $0 \leq i \leq n - 1$ uchun)

- p massivning qiymatlari har xil
- $0 \leq t[i] \leq 10^9$ (barcha $0 \leq i \leq n - 1$ uchun)
- $0 \leq x \leq n - 1$
- $0 \leq y \leq 10^9$

Subtasks

1. (9 ball) $n \leq 2000$
2. (9 ball) n juft. $p[i] = i + 1$ juft i uchun va $p[i] = i - 1$ toq i uchun.
3. (60 ball) $p[i] = i + 1$ barcha $(0 \leq i \leq n - 2)$ uchun va $p[n - 1] = 0$
4. (22 ball) Qo'shimcha chegaralarsiz.

Sample Grader

Namunaviy graderga ma'lumotlarni quyidagi tartibda kiriting:

- qator 1: n q
- qator 2: $p[0]$ $p[1]$ $\dots$ $p[n - 1]$
- qator 3: $t[0]$ $t[1]$ $\dots$ $t[n - 1]$
- qator $4 + k$ ($0 \leq k \leq q - 1$): x y

Namunaviy grader javobni quyidagi tartibda chiqaradi, bu yerda r - `change_time` protsedurasi qaytargan javob:

- qator $1 + k$ ($0 \leq k \leq q - 1$): $r[k]$